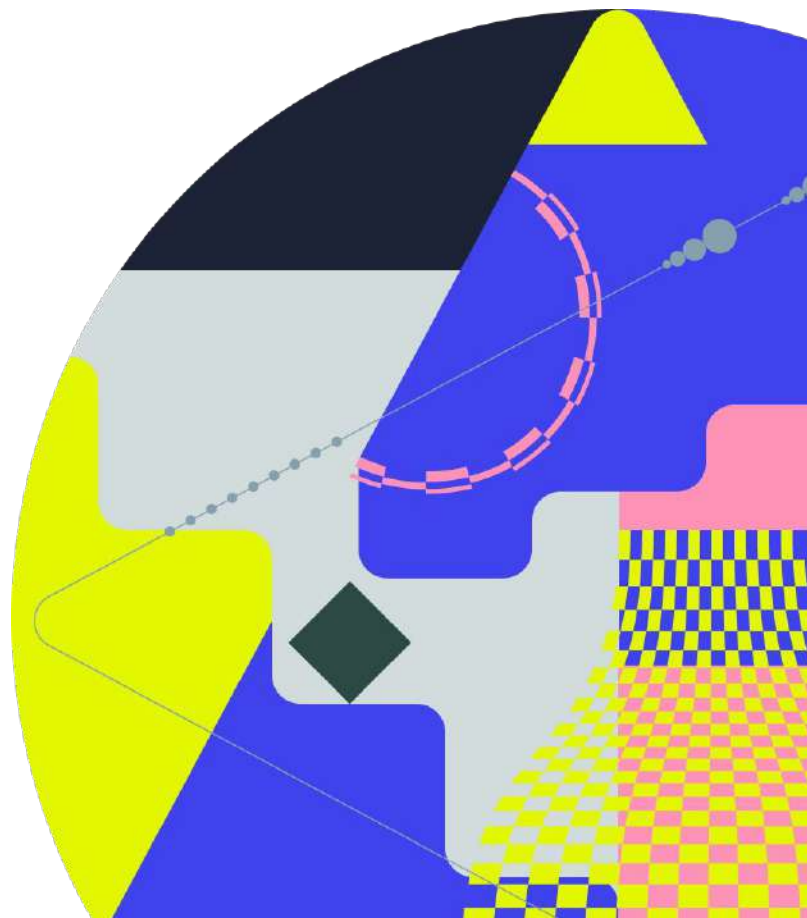


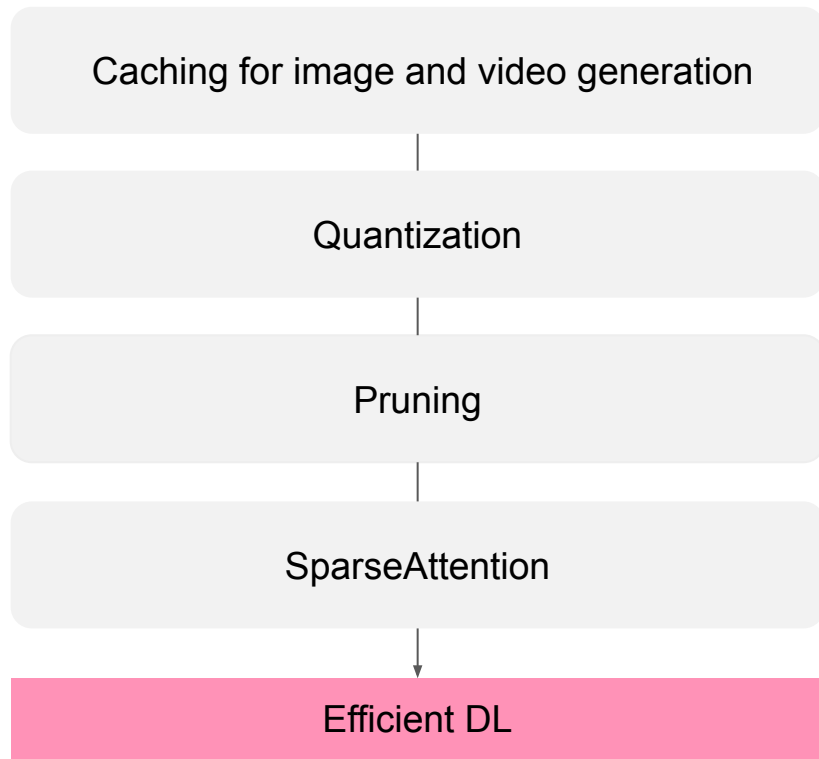
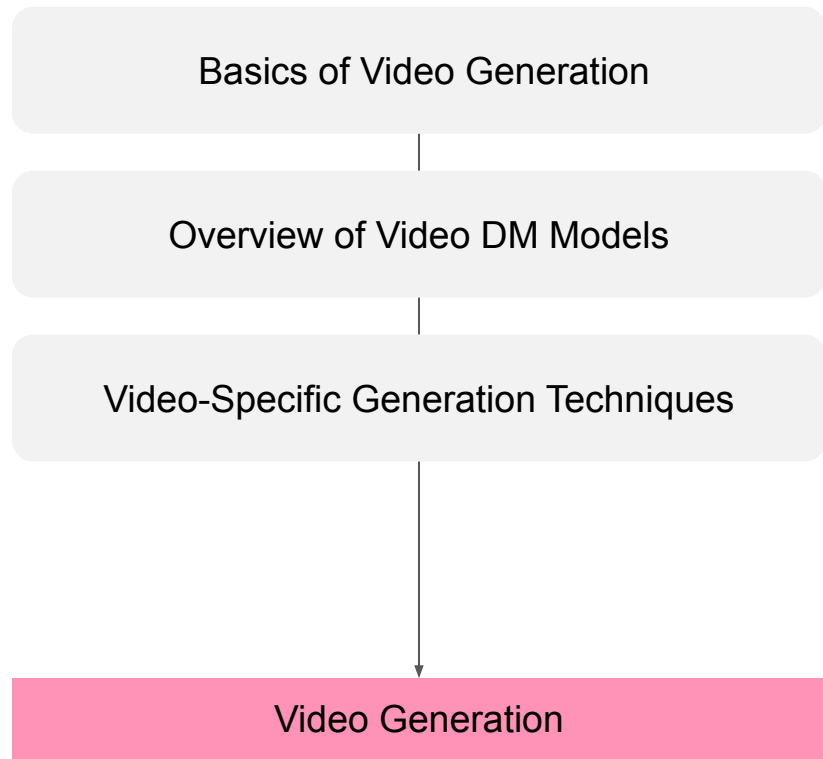
Lectures 9–10

Video Generation Efficient DL for GenAI

Denis Kuznedelev
Yandex Research



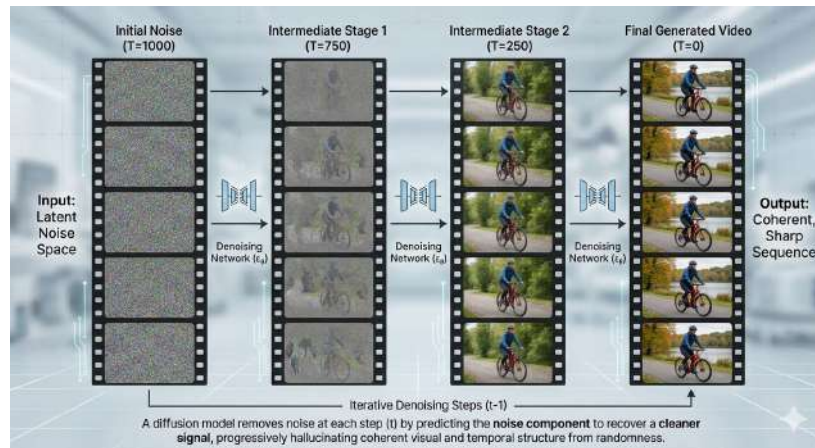
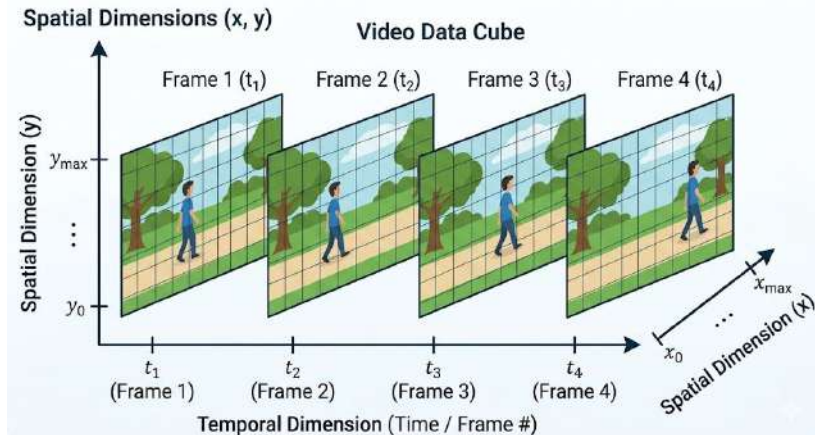
Outline



Video generation process is very similar to image generation.

The main difference is the presence of additional **temporal** dimension.

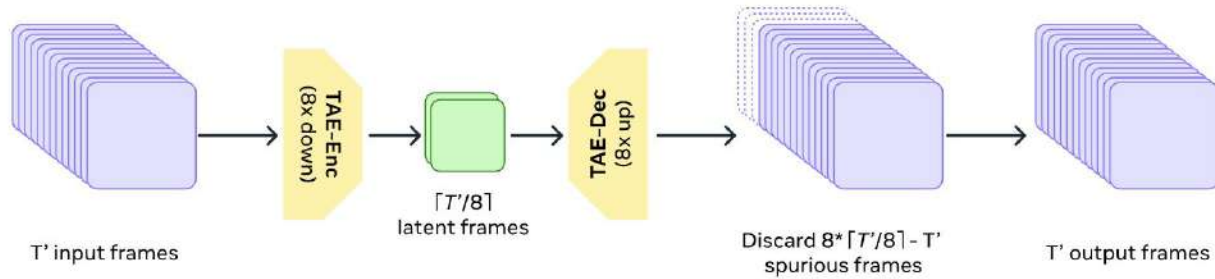
In video generation, we need not only visually plausible individual frames, but also temporal consistency across frames: stable identity, smooth motion, and basic physical plausibility.



Aspect	Image generation	Video generation
Data shape	$H \times W \times C$	$T \times H \times W \times C$
Main requirement	Visual quality	Visual quality + temporal consistency
Common artifacts	blurry objects, wrong anatomy	flicker, identity drift, motion inconsistency
Extra challenge	spatial modeling	spatial + temporal + physical plausibility

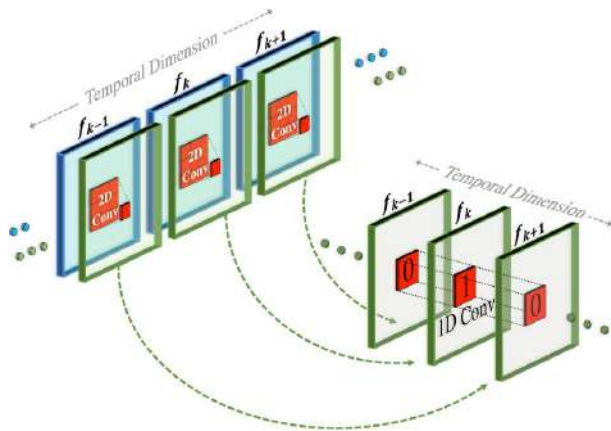
Input video tensor
 $T \times H \times W \times C$

Latent tensor
 $T' \times H' \times W' \times C'$

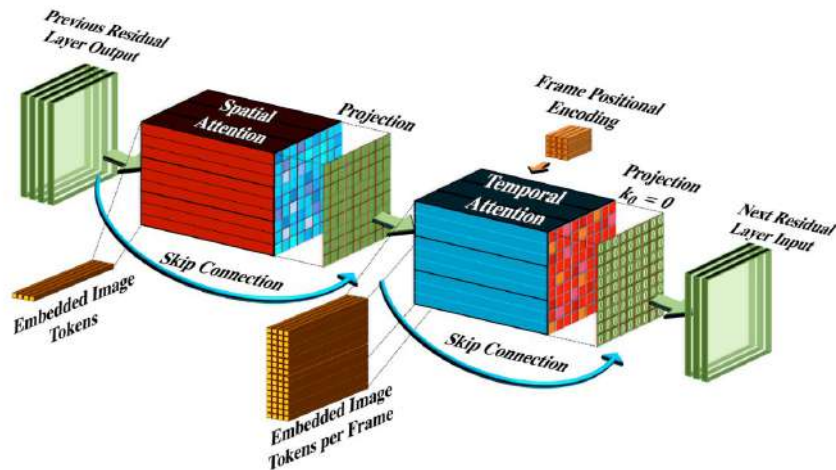


A video VAE compresses not only spatial dimensions, but also the temporal dimension.

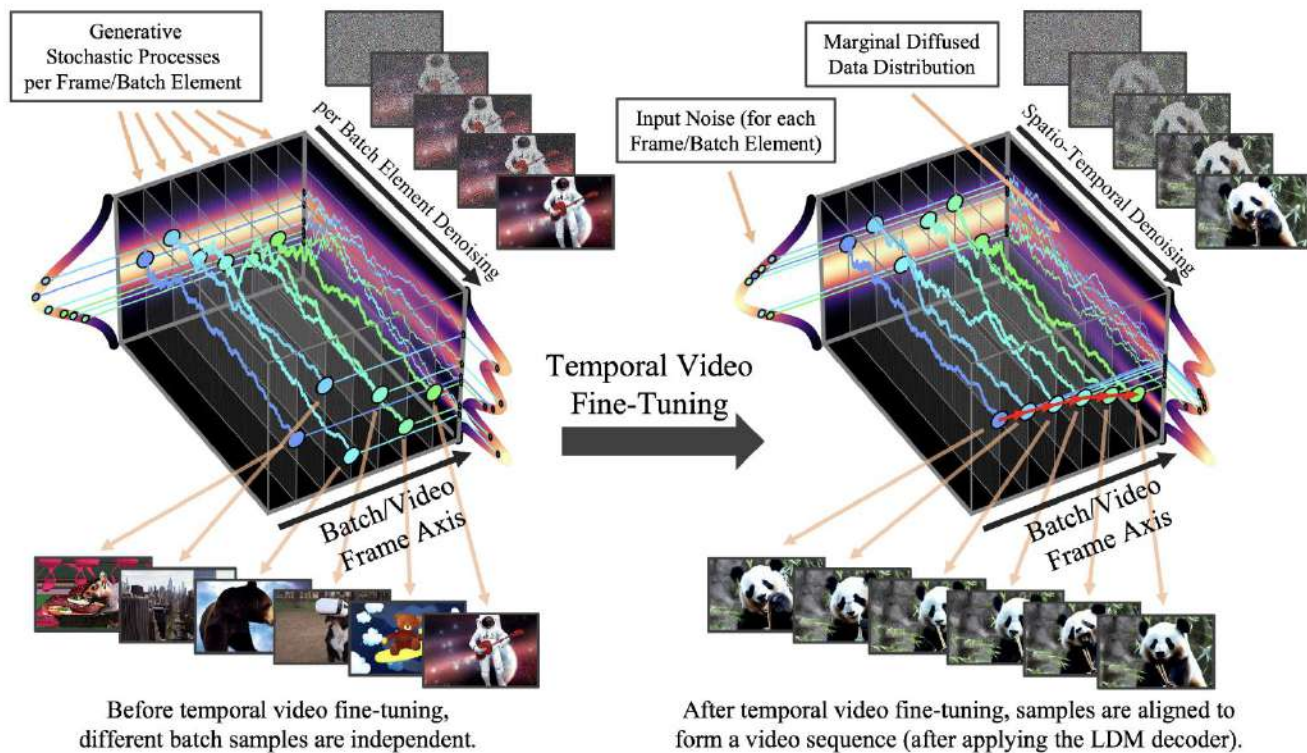
Typical compression rates are 4x/8x.

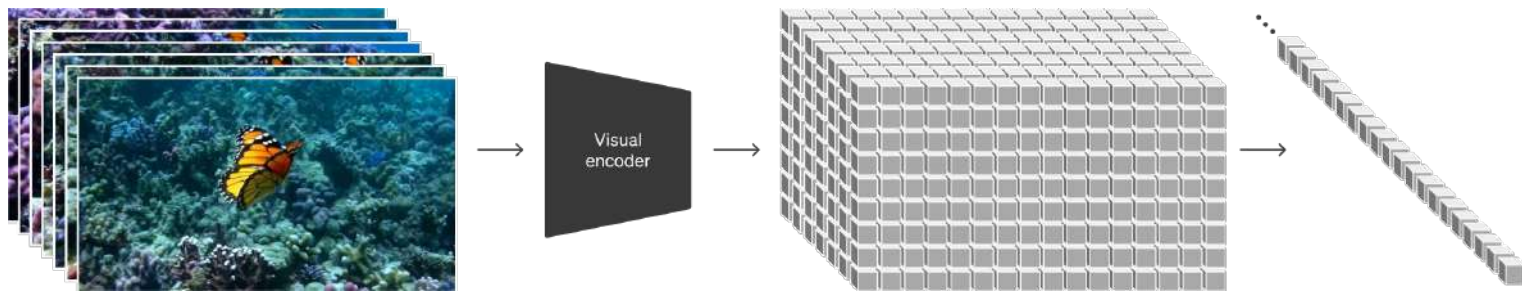


Earlier generation of video diffusion models adopted UNets, initialized from image generators.



2D Convolutions are promoted to 3D. Alternatively, one can add temporal 1D convolution. Temporal attention blocks are added to the model.

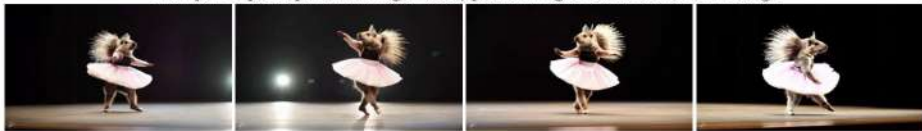




Later work showed that strong results can also be obtained by training a transformer over a flattened sequence of spatiotemporal video tokens.

Text-to-Video

Prompt: A porcupine wearing a tutu, performing a ballet dance on a stage



Prompt: Biker racing through the streets of Los Angeles. Camera tracking shot



Video Personalization and Consistency

Prompt: A person as a scientist performing experiment with test tube

Reference Image



Reference Image



Prompt: A person releases a lantern into the sky



Instruction-Guided Precise Video Editing

Source Video

Edit: Add tinsel streamers to the lantern bottom



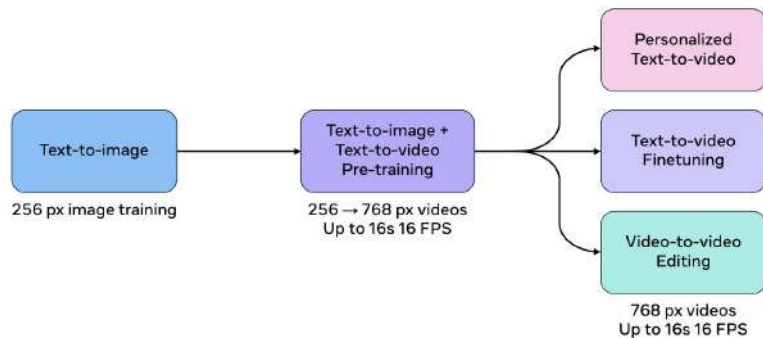
Edit: Transform the lantern into a soaring bubble

Edit: Change background to a city park with a lake

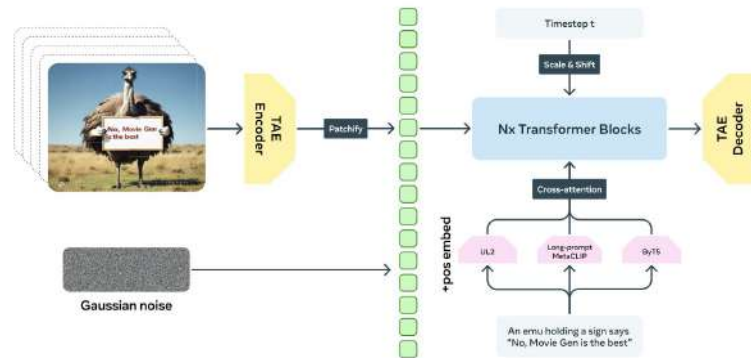


Video Diffusion models may be applied for different tasks.

MovieGen

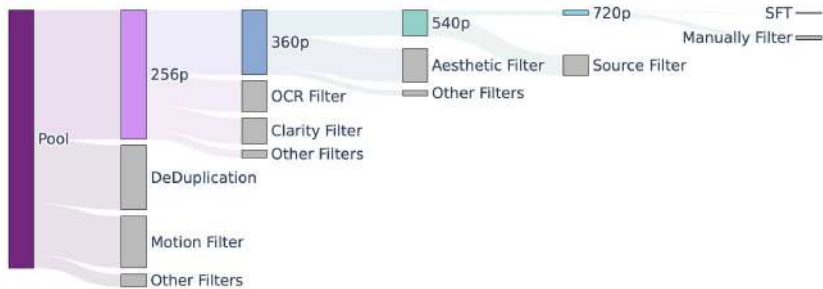


MovieGen paper proposed multistage training pipeline starting from low-resolution images and the switching to mixed image-video training with progressive increase of resolution.

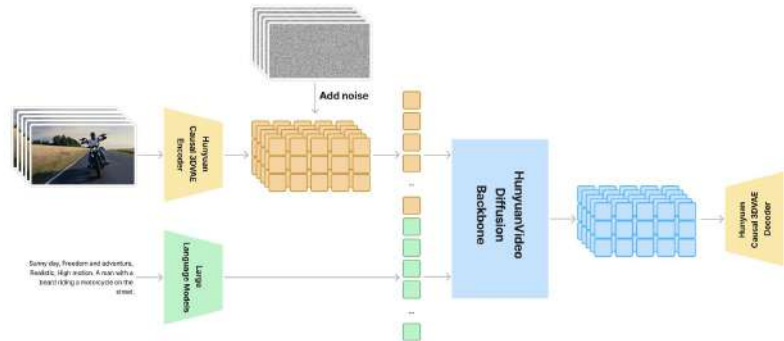


The architecture adopts Llama-like Transformer backbone with conditioning on text encoders.

Hunyuan Video



Hunyuan Video applies filters of progressive strictness on different training stages.



Text and latent tokens sequence are concatenated into a single sequence and processed by MMDiT.

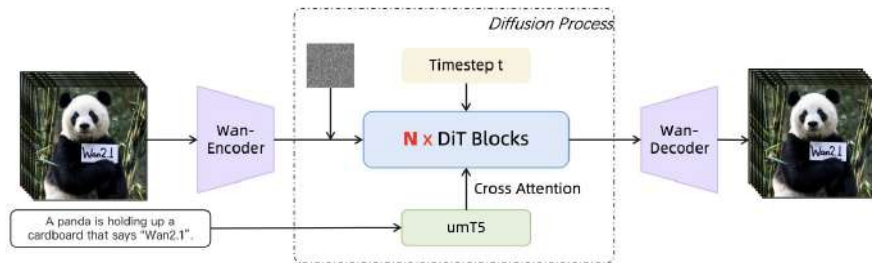
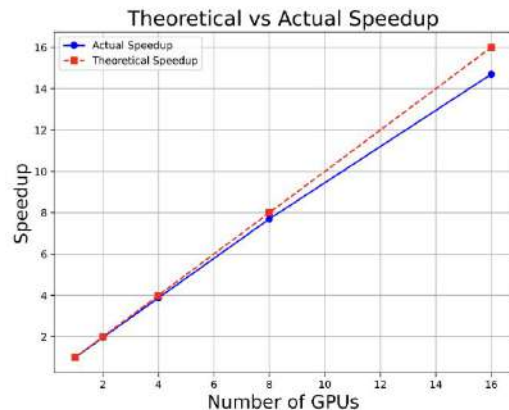


Figure 9: Architecture of the Wan.



Wan is a diffusion transformer with cross attention.

For faster inference Wan applies the following optimizations:

- Context Parallelism
- Diffusion Cache
- FP8 quantization

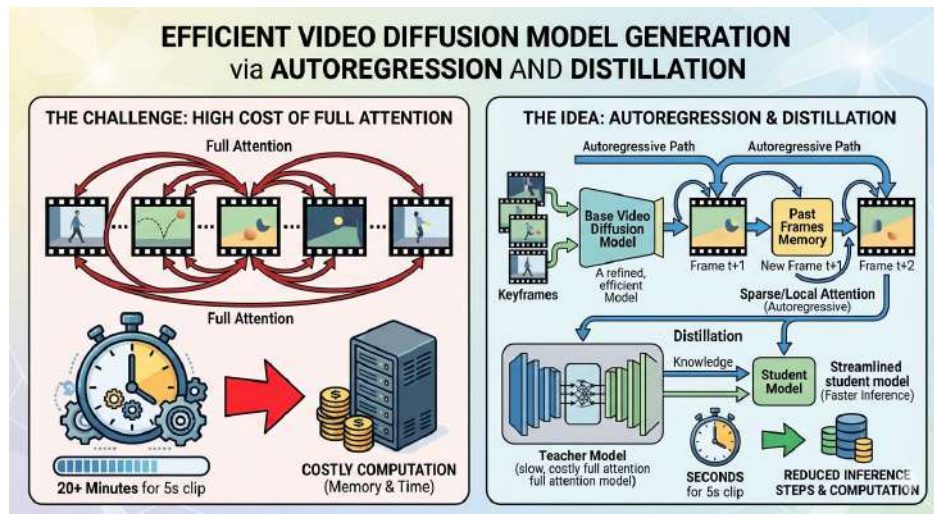


There are numerous labs with strong video generators.
However, they are closed-source.

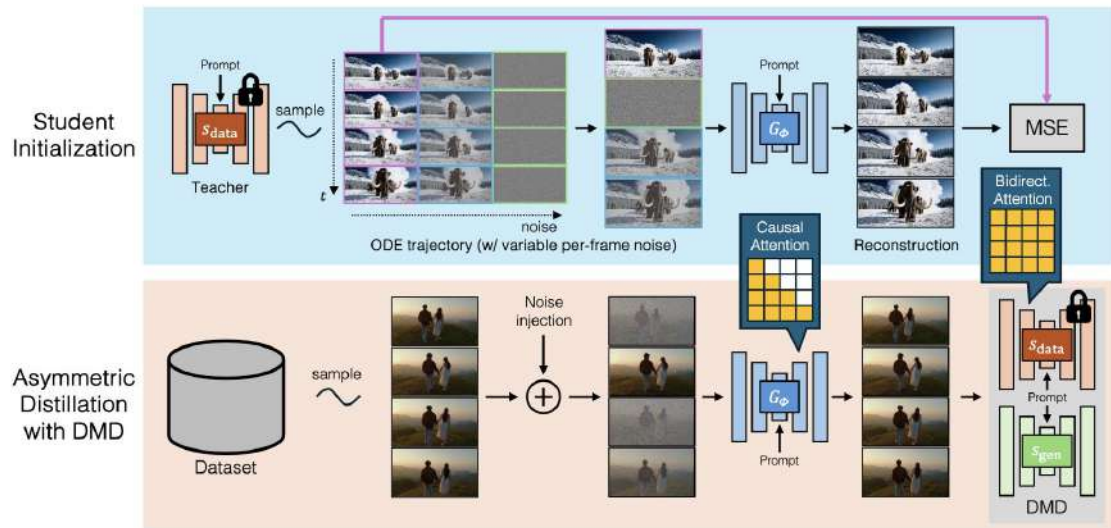
From Slow Bidirectional to Fast Causal Video Generators

Generation process of Video Diffusion models, especially with full attention, is very costly, requiring up to dozens of minutes to generate a short clip. Compute scales quadratically with the number of frames (at fixed spatial resolution).

The idea: generate new frames **autoregressively** chunk-by-chunk while reducing number of inference steps via some known distillation method.



From Slow Bidirectional to Fast Causal Video Generators



CausVid operates as follows:

- 1) Initialize the student with ODE distillation on teacher trajectories
- 2) Distribution distillation between *bidirectional* teacher and *causal* student

From Slow Bidirectional to Fast Causal Video Generators

$$\begin{aligned}\nabla_{\phi} \mathcal{L}_{\text{DMD}} &\triangleq \mathbb{E}_t (\nabla_{\phi} \text{KL}(p_{\text{gen},t} \| p_{\text{data},t})) \\ &\approx -\mathbb{E}_t \left(\int (s_{\text{data}}(\Psi(G_{\phi}(\epsilon), t), t) \right. \\ &\quad \left. - s_{\text{gen}}(\Psi(G_{\phi}(\epsilon), t), t)) \frac{dG_{\phi}(\epsilon)}{d\phi} d\epsilon \right),\end{aligned}\quad (4)$$

$$\mathcal{L}(\theta) = \mathbb{E}_{t, x_0, \epsilon} \|\epsilon_{\theta}(x_t, t) - \epsilon\|_2^2. \quad (2)$$

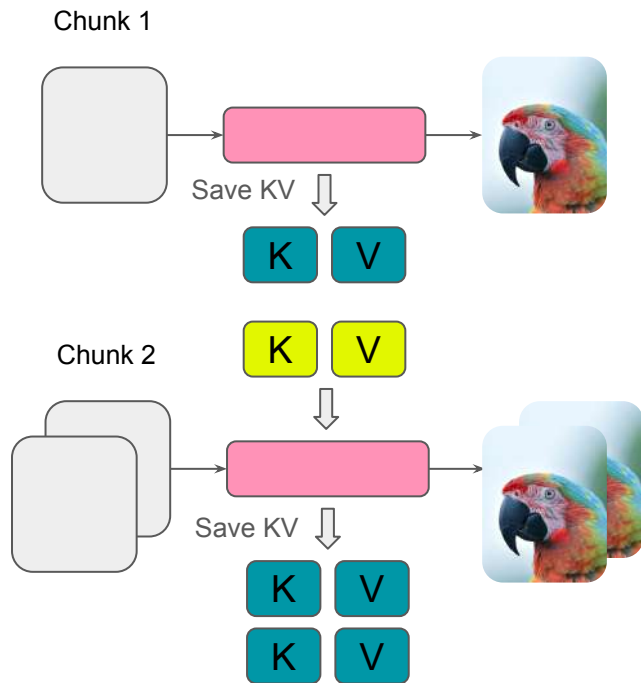
Algorithm 1 Asymmetric Distillation with DMD

Require: Few-step denoising timesteps $\mathcal{T} = \{0, t_1, t_2, \dots, t_Q\}$, video length N , chunk size k , pretrained bidirectional teacher model s_{data} , dataset $\mathcal{D}$.

- 1: **Initialize** student model G_{ϕ} with ODE regression (Sec. 4.3)
 - 2: **Initialize** generator output's score function s_{gen} with s_{data}
 - 3: **while** training **do**
 - 4: **Sample** a video from the dataset and divide frames into $L = \lceil N/k \rceil$ chunks, $\{x_0^i\}_{i=1}^L \sim \mathcal{D}$
 - 5: **Sample** per-chunk timesteps $\{t^i\}_{i=1}^L \sim \text{Uniform}(\mathcal{T})$
 - 6: **Add noise:** $x_{t^i}^i = \alpha_{t^i} x_0^i + \sigma_{t^i} \epsilon^i$, $\epsilon^i \sim \mathcal{N}(0, I)$
 - 7: **Predict** clean frames with the student: $\hat{x}_0^i = G_{\phi}(x_{t^i}^i, t^i)$
 - 8: **Concatenate** predictions: $\hat{x}_0 = [\hat{x}_0^1, \hat{x}_0^2, \dots, \hat{x}_0^L]$
 - 9: **Sample** a single timestep $t \sim \text{Uniform}(0, T)$
 - 10: **Add noise** to predictions: $\hat{x}_t = \alpha_t \hat{x}_0 + \sigma_t \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$
 - 11: **Update** student model G_{ϕ} using DMD loss $\mathcal{L}_{\text{DMD}} \triangleright$ Eq. 4
 - 12: **Train** generator's score function s_{gen} :
 - 13: **Sample new noise** $\epsilon' \sim \mathcal{N}(0, I)$
 - 14: **Generate noisy** $\hat{x}_t$: $\hat{x}_t = \alpha_t \hat{x}_0 + \sigma_t \epsilon'$
 - 15: Compute denoising loss and update s_{gen} . $\triangleright$ Eq. 2
 - 16: **end while**
-

The training procedure

From Slow Bidirectional to Fast Causal Video Generators



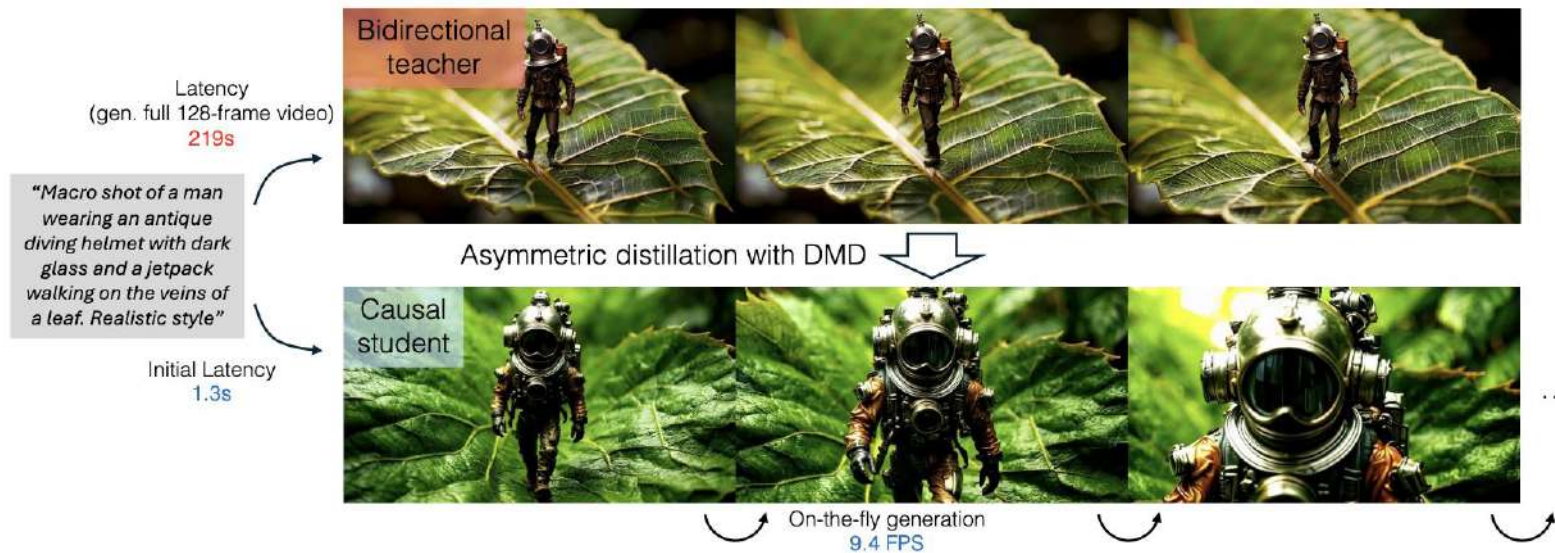
Algorithm 2 Inference Procedure with KV Caching

Require: Denoising timesteps $\{t_0 = 0, t_1, \dots, t_Q\}$, video length N , chunk size k , few-step autoregressive video generator G_ϕ ,

- 1: Divide frames into $L = \lceil N/k \rceil$ chunks
- 2: **Initialize** KV cache $C \leftarrow \emptyset$
- 3: **for** $i = 1$ to L **do**
- 4: **Initialize current chunk:** $x_{t_Q}^i \sim \mathcal{N}(0, I)$
- 5: **Iterative denoising over timesteps:**
- 6: **for** $j = Q$ to 1 **do**
- 7: **Generate output:** $\hat{x}_{t_j}^i = G_\phi(x_{t_j}^i, t_j)$ using cache C
- 8: **Update chunk:** $x_{t_{j-1}}^i = \alpha_{t_{j-1}} \hat{x}_{t_j}^i + \sigma_{t_{j-1}} \epsilon'$
- 9: **end for**
- 10: **Update KV cache:**
- 11: Compute KV pairs with a forward pass $G_\phi(x_0^i, 0)$
- 12: Append new KV pairs to cache C
- 13: **end for**
- 14: **Return** $\{x_0^i\}_{i=1}^L$

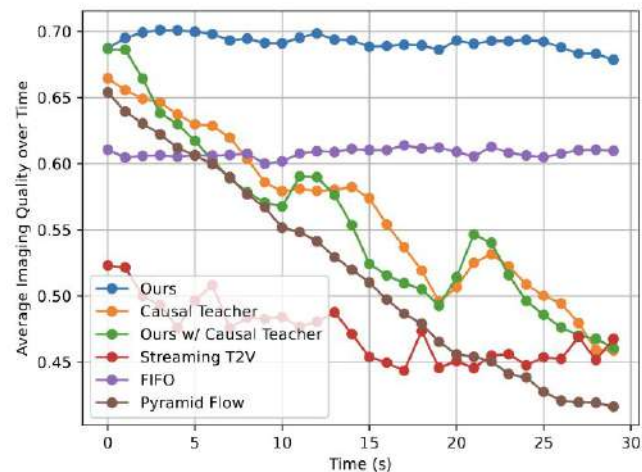
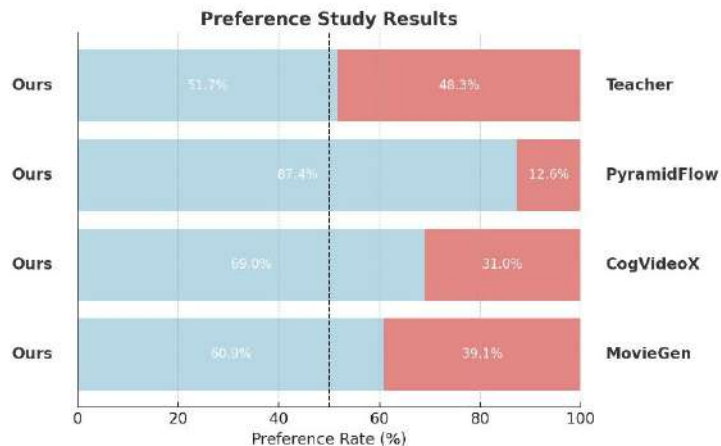
CausVid generates current chunk and saves computed KV for further reuse

From Slow Bidirectional to Fast Causal Video Generators



CausVid allows for streaming generation when short chunks of Video are generated on the fly

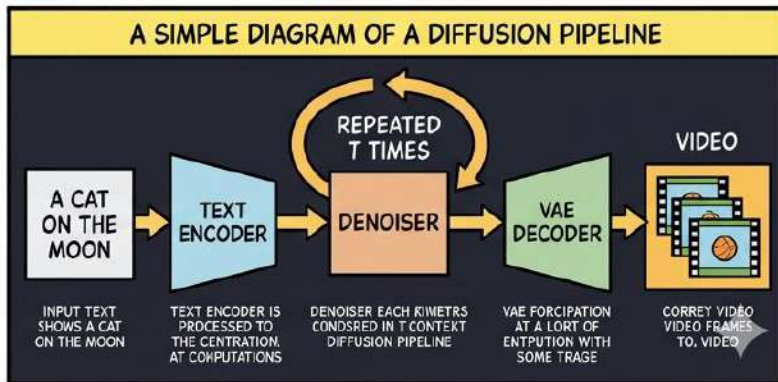
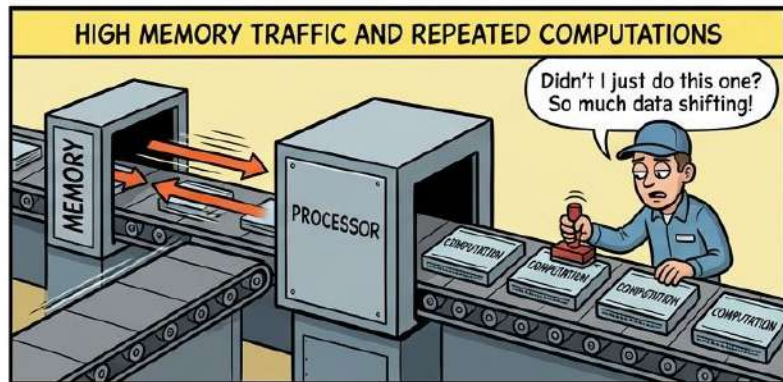
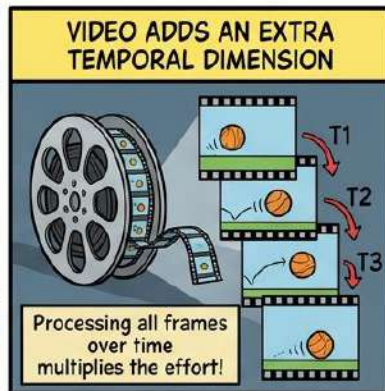
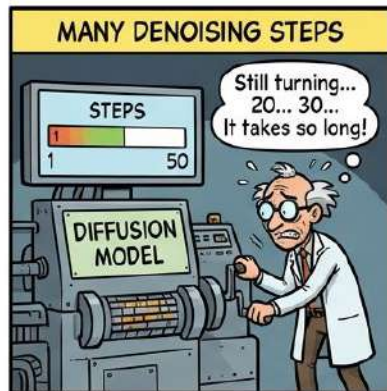
From Slow Bidirectional to Fast Causal Video Generators



CausVid maintains quality close to the bidirectional teacher while substantially improving generation speed.

CausVid maintains decent video quality even for longer generations

WHERE DOES INFERENCE TIME GO IN IMAGE/VIDEO GENERATION?



Image/Video Generation process
is costly



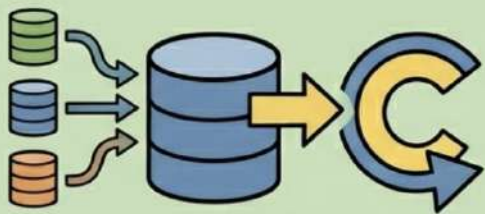
Step distillation allows one to drastically
reduce inference cost



There exist alternative strategies

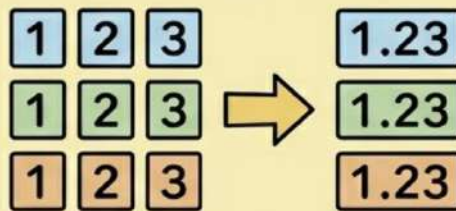
Efficiency Taxonomy

A. REUSE WORK



Caching Activations
Across Denoising Steps

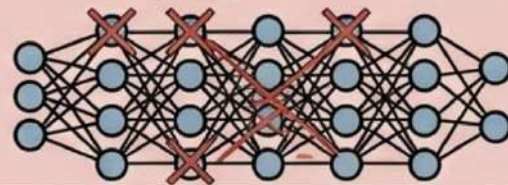
B. USE CHEAPER ARITHMETIC



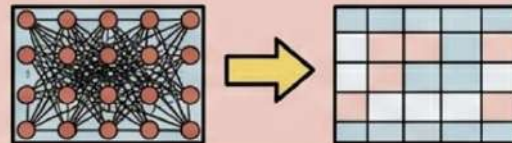
Quantization of Weights /
Activations / Attention

C. DO LESS WORK

Pruning Layers or Blocks



Sparse Attention instead
of Dense Attention



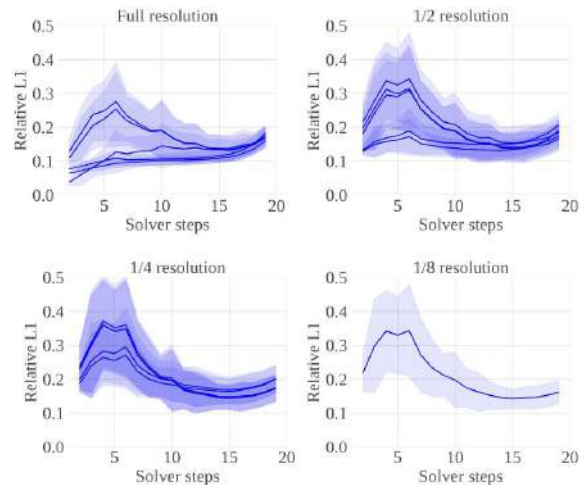


What if I say that one can reuse computation results
from previous denoising steps?

Cache Me if You Can: Accelerating Diffusion Models through Block Caching

SpatialTransformer blocks perform costly self- and cross- attention operations

Empirical observations show that activations in these blocks evolve smoothly

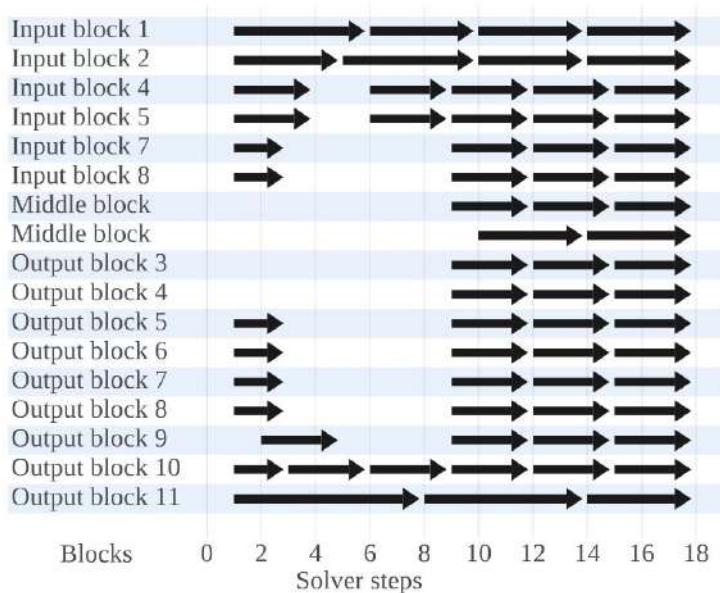


$$L1_{\text{rel}}(i, t) = \frac{\|C_i(x_t, s_t) - C_i(x_{t-1}, s_{t-1})\|_1}{\|C_i(x_t, s_t)\|_1}$$

Cache Me if You Can: Accelerating Diffusion Models through Block Caching

Idea: reuse activations computed on previous timestep

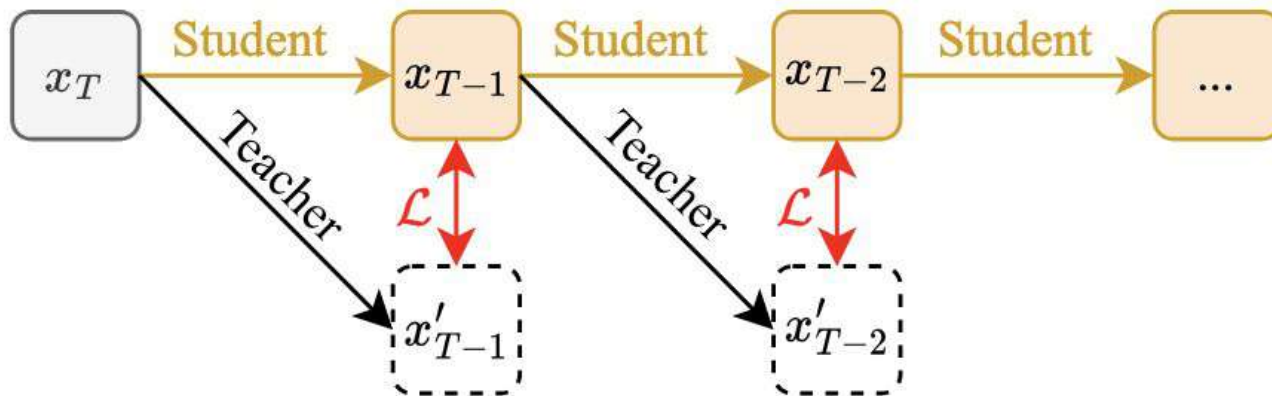
The proposed approach is validated on UNet architecture (EMU by Meta*)



Arrows correspond to cached timesteps

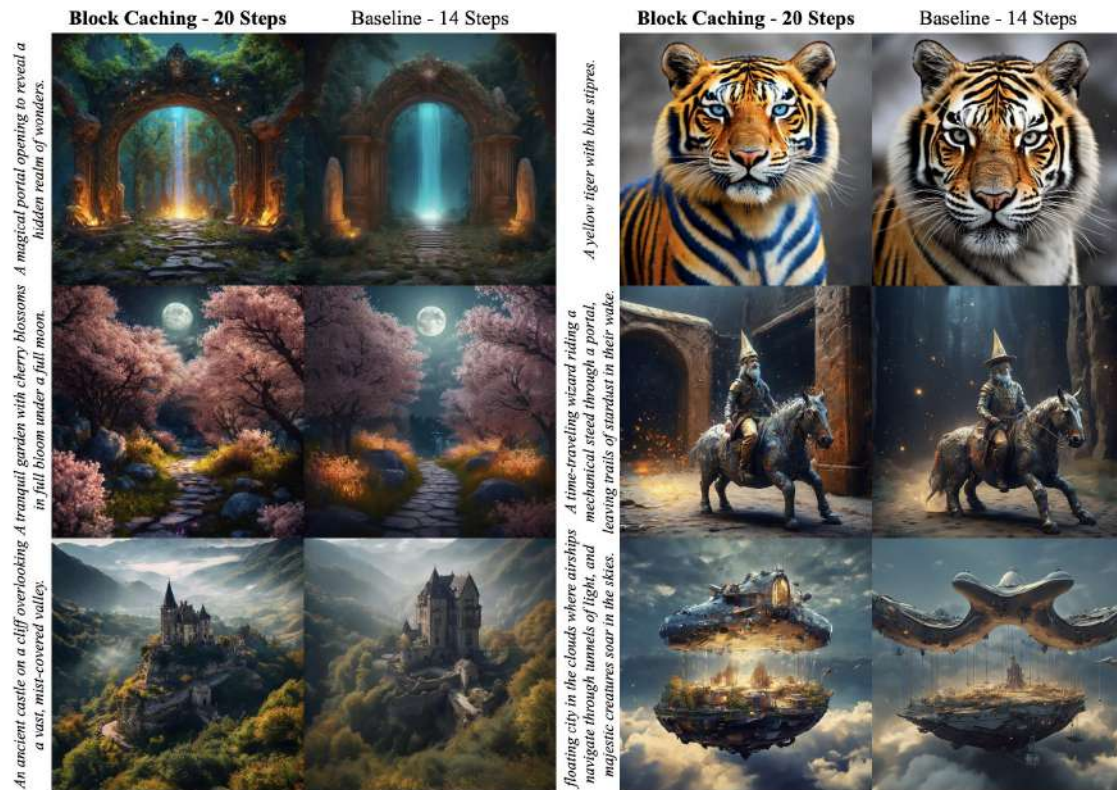
*extremist organization banned in Russia

Cache Me if You Can: Accelerating Diffusion Models through Block Caching



Block Caching works decently,
but the best results are achieved with distillation

Cache Me if You Can: Accelerating Diffusion Models through Block Caching



With identical inference speed Block Caching offers better fidelity compared to naive step reduction

Cache Me if You Can: Accelerating Diffusion Models through Block Caching

One can achieve speed-ups of order 40-80% with block caching

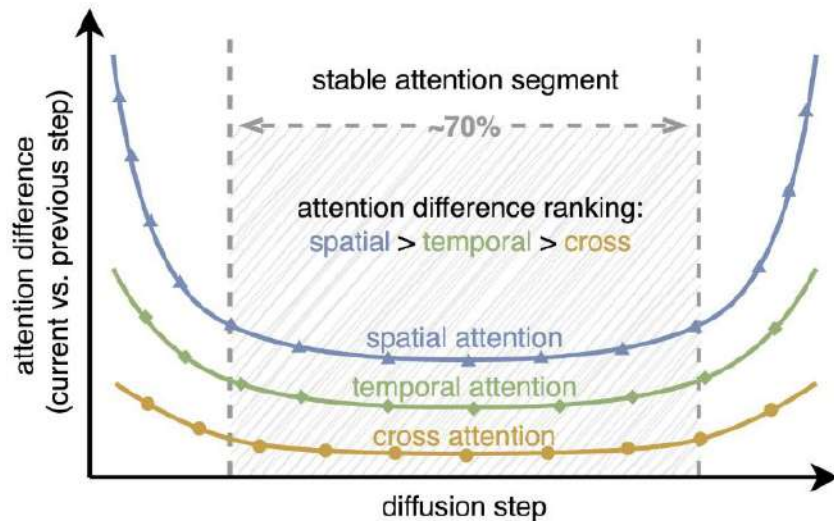
Solver	Steps	Caching	SS	FID ↓	Img/s ↑	Speedup ↑
DPM [27]	20			<u>17.15</u>	2.17	1.00×
	14			18.67	3.10	1.43×
	20	✓		17.58	3.64	1.68×
	20	✓	✓	15.95	<u>3.59</u>	<u>1.65×</u>
DDIM [48]	20			17.43	2.17	1.00×
	14			17.11	3.10	1.43×
	20	✓		<u>16.52</u>	3.48	1.60×
	20	✓	✓	16.02	<u>3.45</u>	<u>1.58×</u>
DPM [27]	50			17.44	0.87	1.00×
	30			<u>17.21</u>	1.46	1.67×
	50	✓		17.23	1.61	1.85×
	50	✓	✓	15.18	<u>1.59</u>	<u>1.82×</u>
DDIM [48]	50			17.76	0.87	1.00×
	30			17.42	1.46	1.67×
	50	✓		<u>16.65</u>	1.59	1.82×
	50	✓	✓	15.15	<u>1.56</u>	<u>1.79×</u>

Real-Time Video Generation with Pyramid Attention Broadcast

Idea of activation caching is applicable to Video Diffusion models as well

Different kinds of attention exhibit different variability across timesteps

Idea: cache cross attentions more aggressively



Real-Time Video Generation with Pyramid Attention Broadcast



PAB yields generations very close to baseline generation

Real-Time Video Generation with Pyramid Attention Broadcast

model	method	VBench $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	SSIM $\uparrow$	FLOPs (T) $\downarrow$	latency (s) $\downarrow$	speedup
Open-Sora	original	79.22	—	—	—	3230.24	26.54	—
	Δ -DiT	78.21	11.91	0.5692	0.4811	3166.47	25.87	1.03 $\times$
	T-GATE	77.61	15.50	0.3495	0.6760	2818.40	22.22	1.19 $\times$
	PAB₂₄₆	78.51	27.04	0.0925	0.8847	2657.70	19.87	1.34 $\times$
	PAB₃₅₇	77.64	24.50	0.1471	0.8405	2615.15	19.35	1.37 $\times$
	PAB₅₇₉	76.95	23.58	0.1743	0.8220	2558.25	18.52	1.43$\times$
Open-Sora-Plan	original	80.39	—	—	—	12032.40	46.49	—
	Δ -DiT	77.55	13.85	0.5388	0.3736	12027.72	46.08	1.01 $\times$
	T-GATE	80.15	18.32	0.3066	0.6219	10663.32	39.37	1.18 $\times$
	PAB₂₄₆	80.30	18.80	0.3059	0.6550	9276.57	33.83	1.37 $\times$
	PAB₃₅₇	77.54	16.40	0.4490	0.5440	8899.32	31.61	1.47 $\times$
	PAB₅₇₉	71.81	15.47	0.5499	0.4717	8551.26	29.50	1.58$\times$
Latte	original	77.40	—	—	—	3439.47	11.18	—
	Δ -DiT	52.00	8.65	0.8513	0.1078	3437.33	10.85	1.02 $\times$
	T-GATE	75.42	19.55	0.2612	0.6927	3059.02	9.88	1.13 $\times$
	PAB₂₃₅	76.32	19.71	0.2699	0.7014	2767.22	8.91	1.25 $\times$
	PAB₃₄₇	73.69	18.07	0.3517	0.6582	2648.45	8.45	1.32 $\times$
	PAB₄₆₉	73.13	17.16	0.3903	0.6421	2576.77	8.21	1.36$\times$

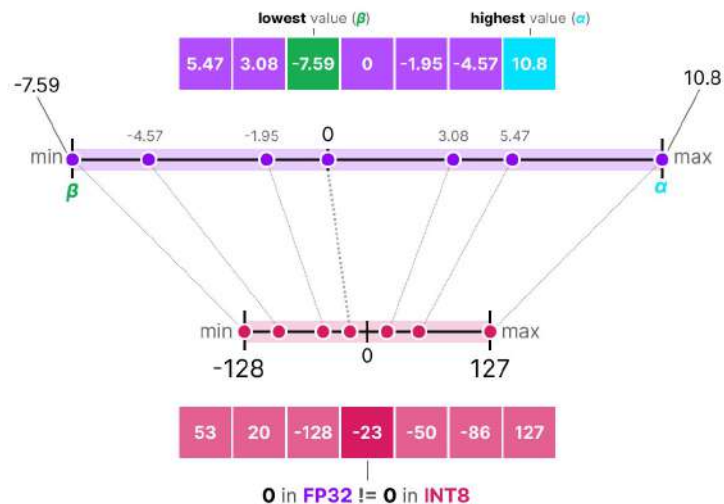
Speed-ups are of order 30-60%

Quantization for Diffusion Models

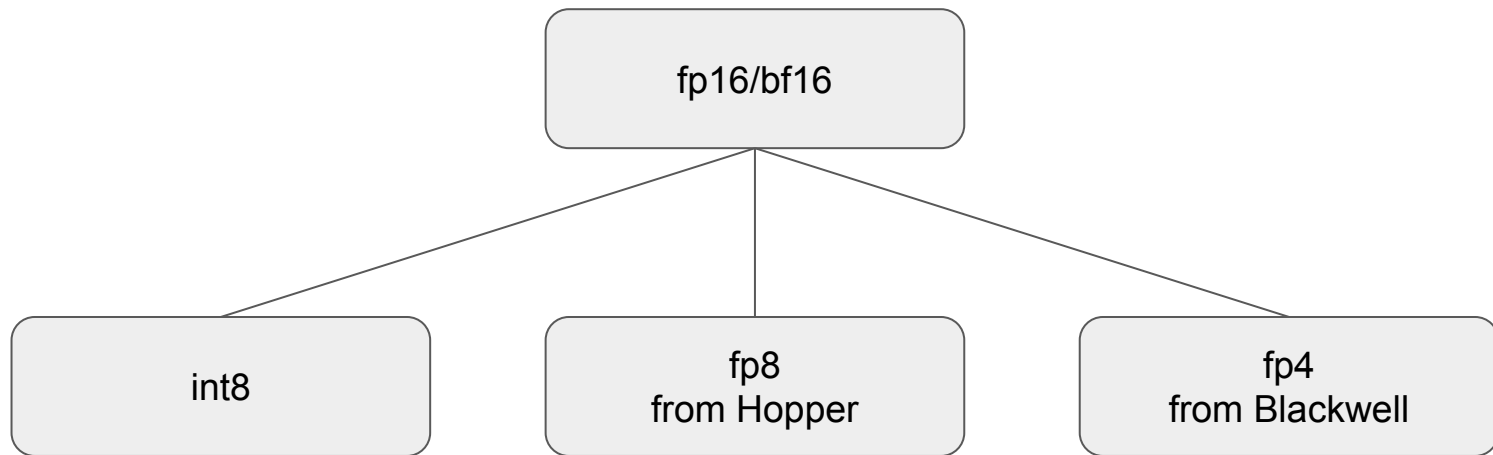
Quantization is a way to reduce the amount of memory required to store an array of data by representing its elements from a limited set.

Quantization is a general purpose technique for **model compression** and **inference acceleration**.

Computation in lower precision may be cheaper and faster.



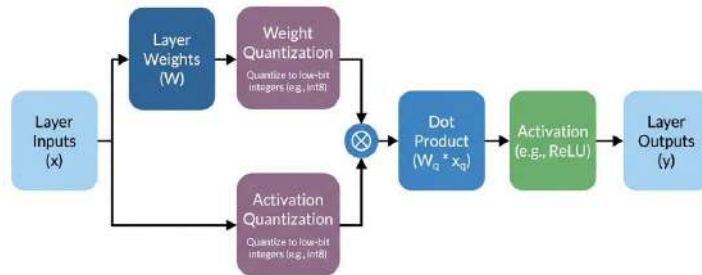
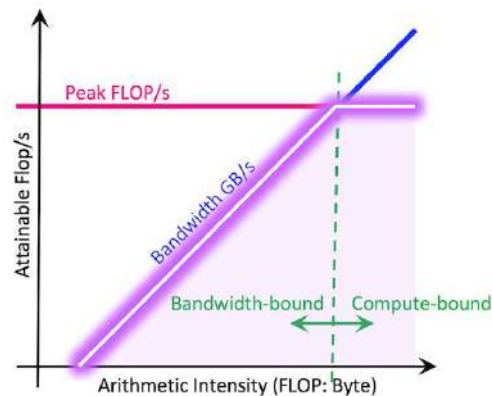
Quantization for Diffusion Models



Quantization for Diffusion Models

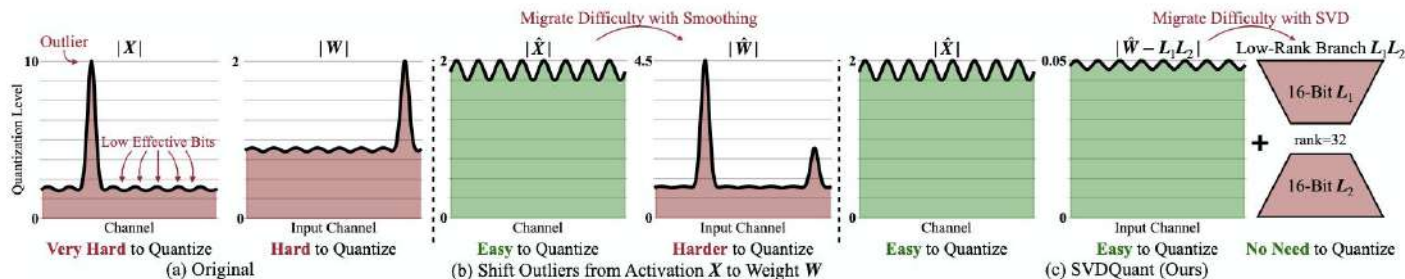
Generative model inference is usually compute-bound (e.g latency is dominated by computation)

Therefore, **weight** and **activation** quantization is needed to accelerate inference.



SVDQuant: Absorbing Outliers by Low-Rank Components for 4-Bit Diffusion Models

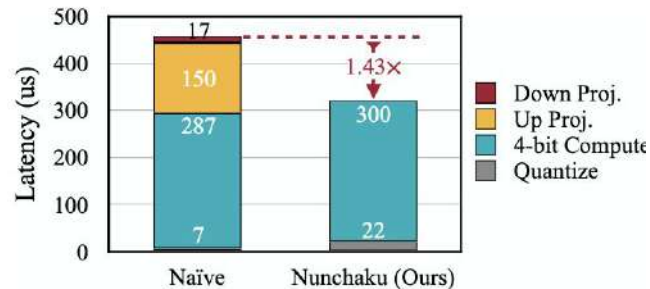
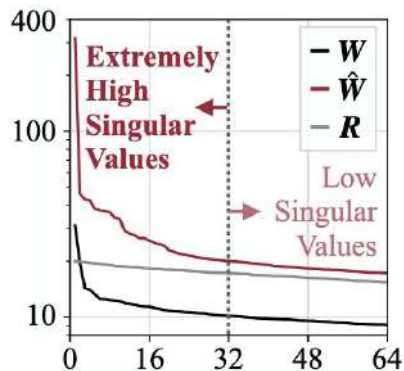
SVDQuant proposed to represent weight as a sum of quantized + low-rank component



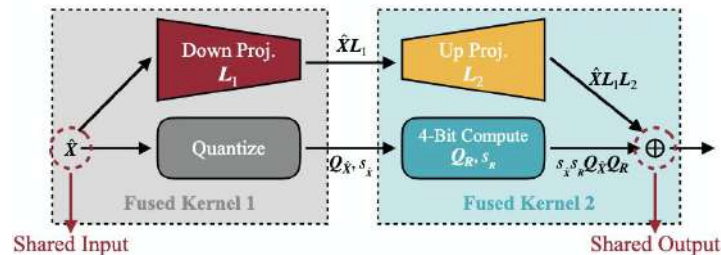
$$XW = \hat{X}\hat{W} = \hat{X}L_1L_2 + \hat{X}R \approx \underbrace{\hat{X}L_1L_2}_{\text{16-bit low-rank branch}} + \underbrace{Q(\hat{X})Q(R)}_{\text{4-bit residual}}.$$

SVDQuant: Absorbing Outliers by Low-Rank Components for 4-Bit Diffusion Models

SVDQuant fuses LoRA adapter into a single kernel with quantized computation branch



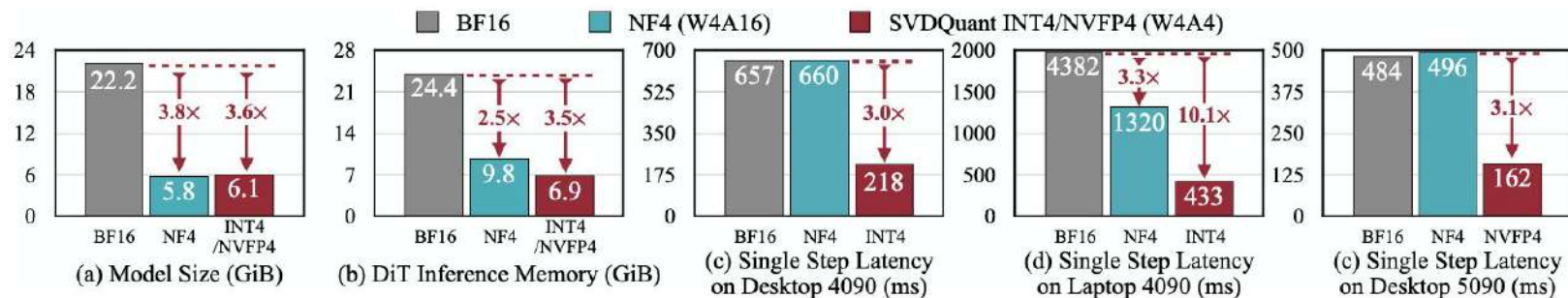
(a) Latency Breakdown on QKV projection



(b) Nunchaku Kernel Fusion

SVDQuant: Absorbing Outliers by Low-Rank Components for 4-Bit Diffusion Models

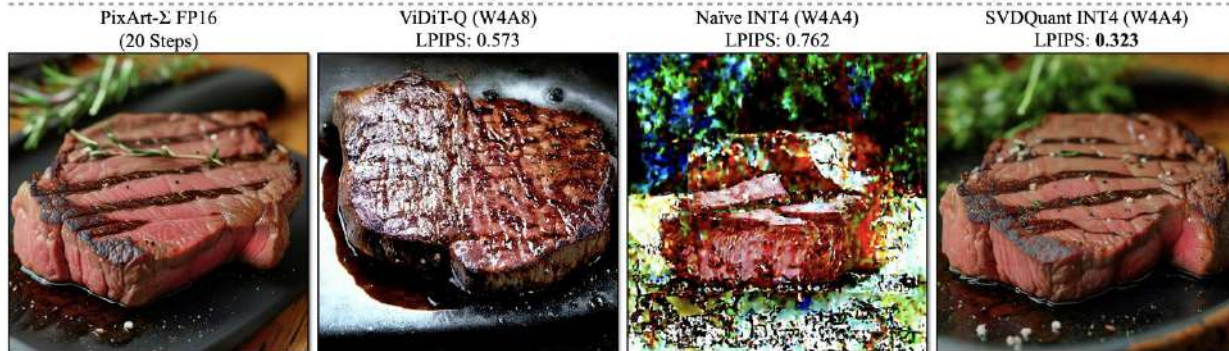
SVDQuant allows one to reduce memory consumption and accelerate inference



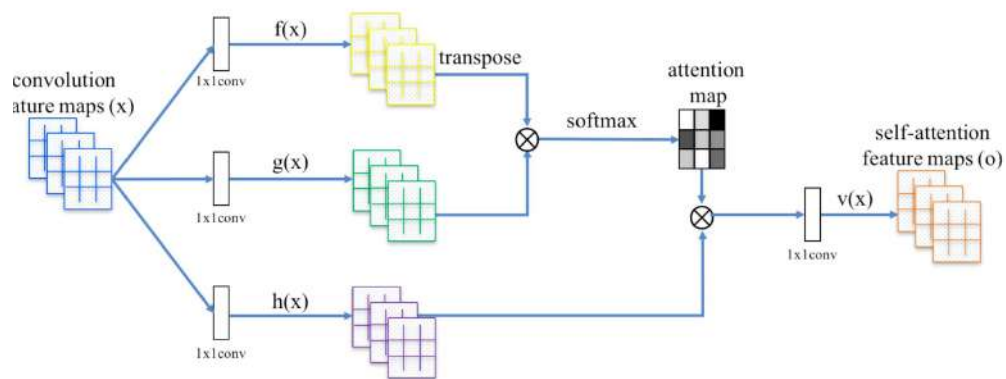
SVDQuant: Absorbing Outliers by Low-Rank Components for 4-Bit Diffusion Models



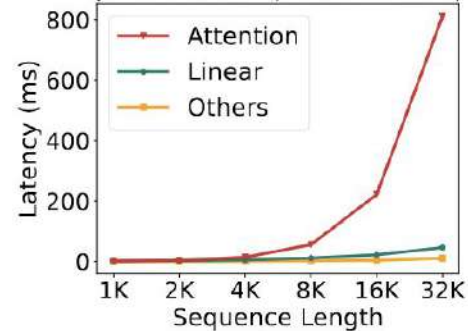
Prompt: a cyberpunk cat holding a huge neon sign that says "SVDQuant is lite and fast", wearing fancy goggles and a black leather jacket.



Prompt: medium rare steak tenderloin super tasty photo.



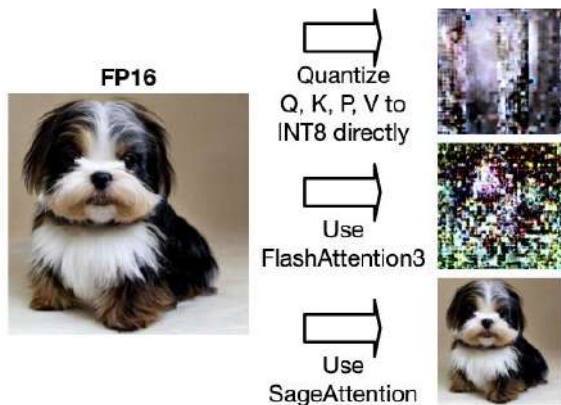
Latency of operators in a transformer model
(NumHead=32, HeadDim=64)



Attention operation
(especially for long contexts)
requires large costly matmuls ($Q K^T$).

One can make inference faster via
performing attention in lower precision.

SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration



$$\gamma(K) = K - \text{mean}(K)$$

$$\sigma(q(K - \text{mean}(K)^\top)) = \sigma(qK^\top - q \cdot \text{mean}(K)) = \sigma(qK^\top)$$

Quantization (Q, K)	Smoothing K	Llama WikiText ↓	CogVideo (Fscore) ↑	Unidiffuser (FID) ↓	UltraPixel (FID) ↓	TIMM ImageNet ↑
Full-Precision	-	5.823	3.768	163.33	179.78	84.79%
Per-token	✗	5.824	1.924	221.18	193.36	84.21%
	✓	5.824	3.734	166.52	179.79	84.74%
Per-block	✗	5.825	2.014	229.08	195.67	84.18%
	✓	5.824	3.718	166.93	179.98	84.76%
Per-tensor	✗	5.826	1.902	267.06	196.26	84.12%
	✓	5.824	3.640	167.65	180.21	84.69%
FlashAttn3 (with quant)		5.850	3.394	394.13	383.61	84.70%

Naive quantization leads to poor quality.

The core idea of SageAttention is to
smooth keys prior to quantization

SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration

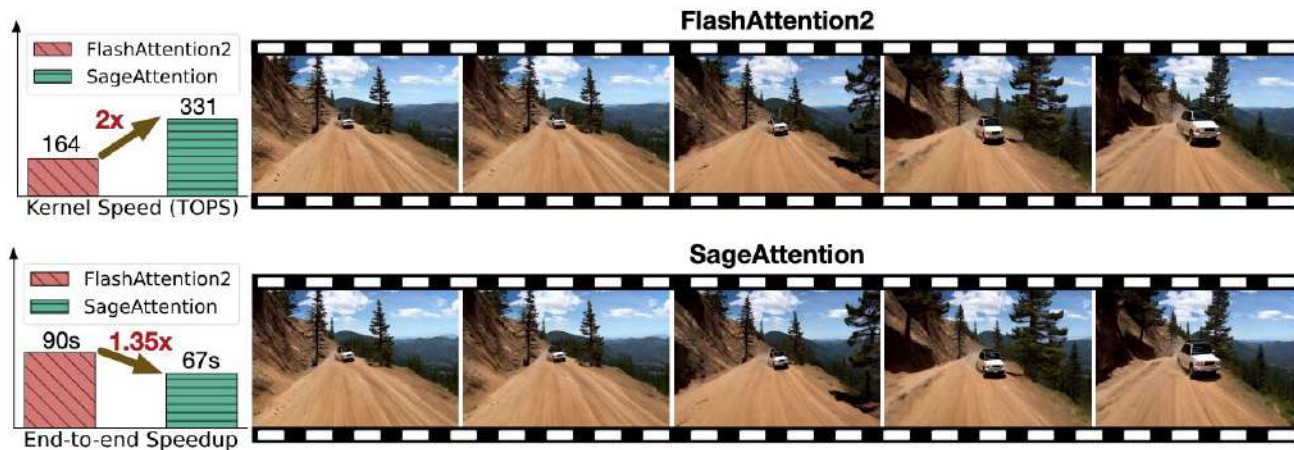


Figure 1: An example of SageAttention on video generation (CogvideoX on RTX4090).

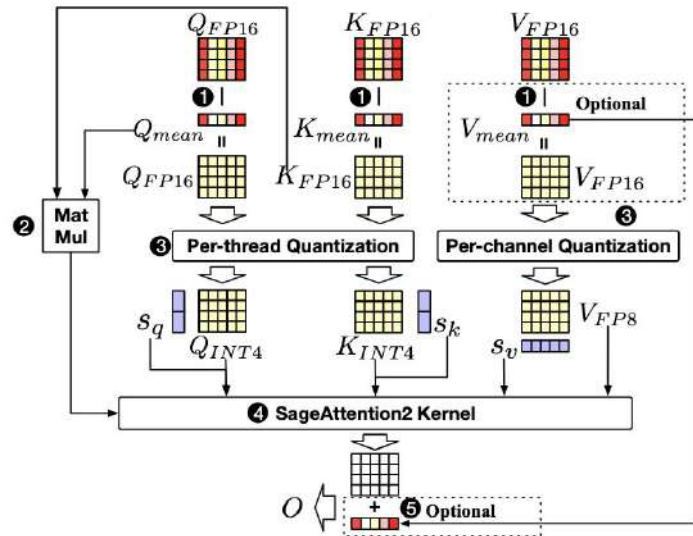
SageAttention provides faster inference via quantizing keys & queries to 8 bits without affecting generation quality

SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization

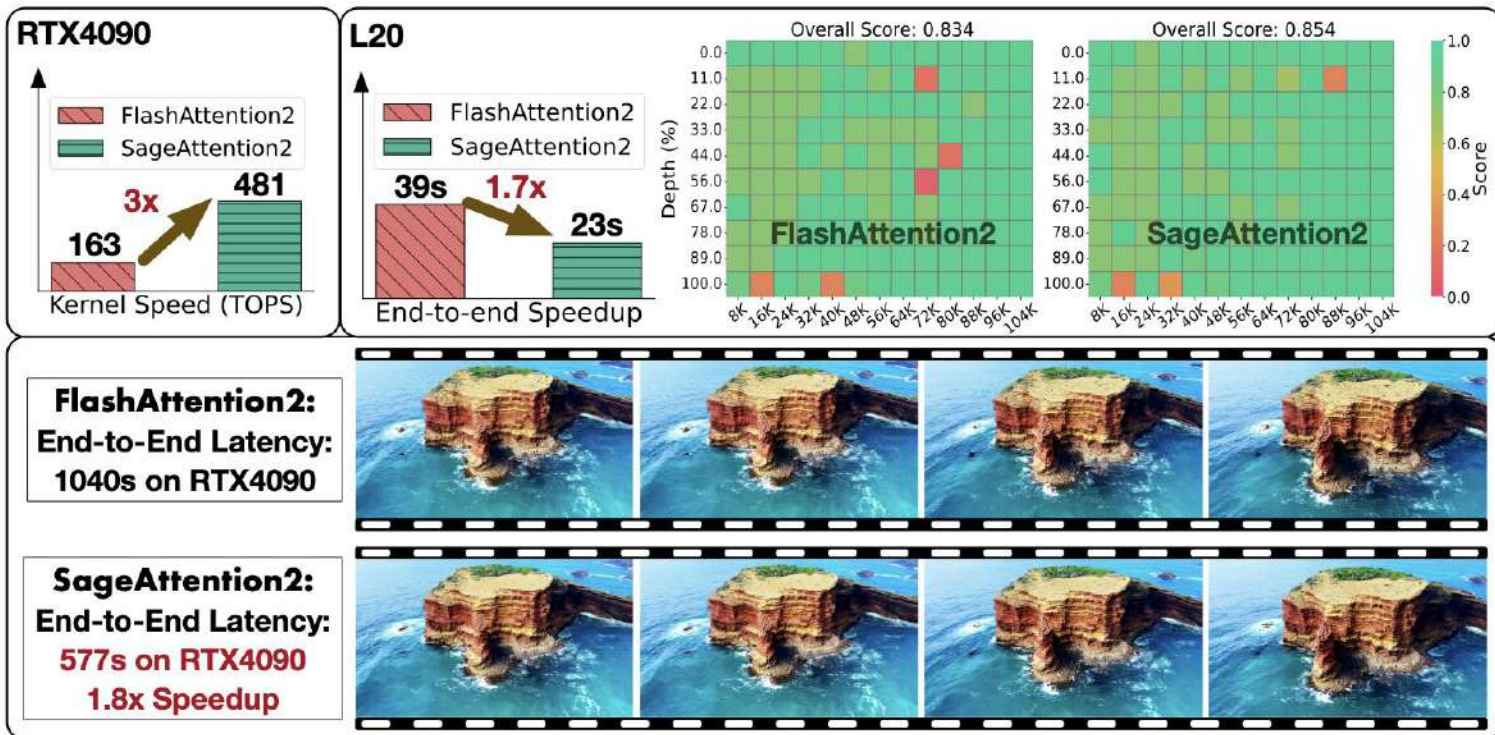
SageAttention2 goes further with quantizing keys & queries to 4 bit

$$\gamma(Q_i) = Q_i - \bar{q}_i, \quad \gamma(K_j) = K_j - \bar{k}_j,$$

Both key and query smoothing together with increasing quantization granularity are needed to maintain quality



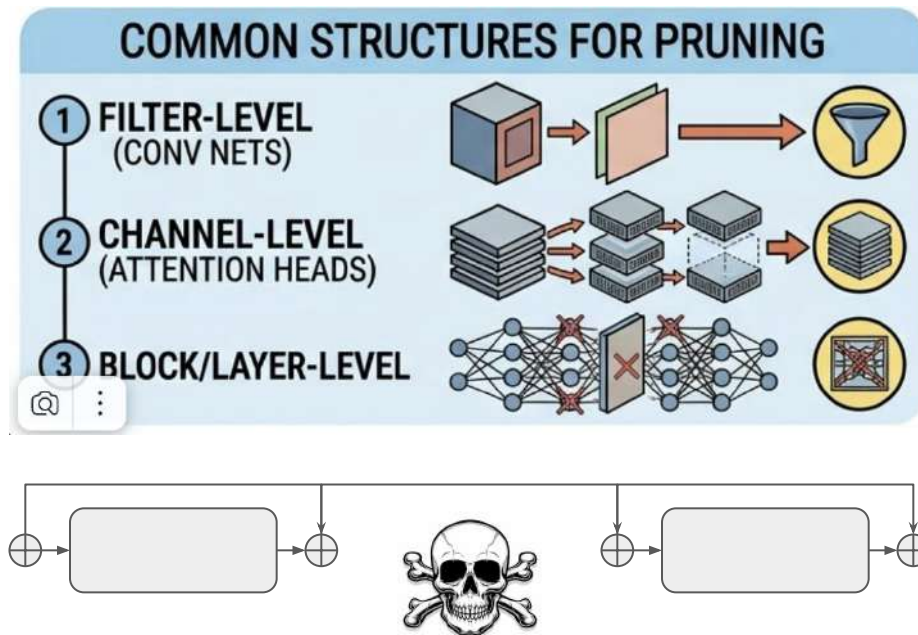
SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization



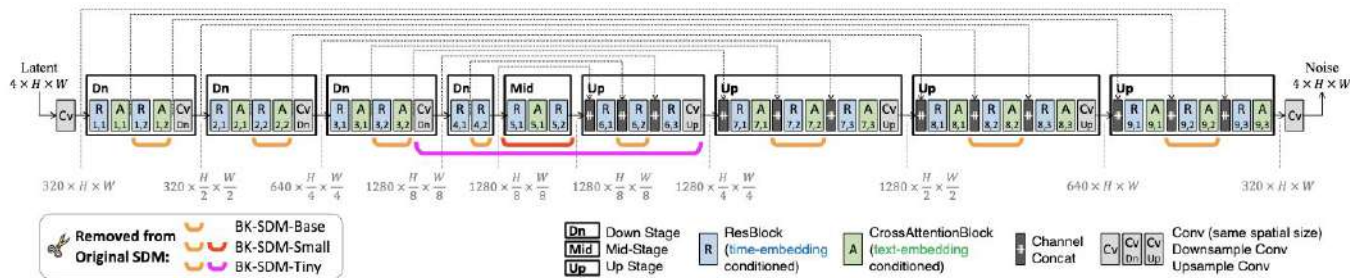
Structured Sparsity

A common approach for inference latency reduction is pruning (removal) of the least significant network components.

One can remove channels/dimensions of weight tensors or skip residual blocks.



Structured Sparsity



Some parts of the denoiser network may be removed almost without performance drop or one can recover after short tuning.

SDM-v1.4

No Mid-Stage

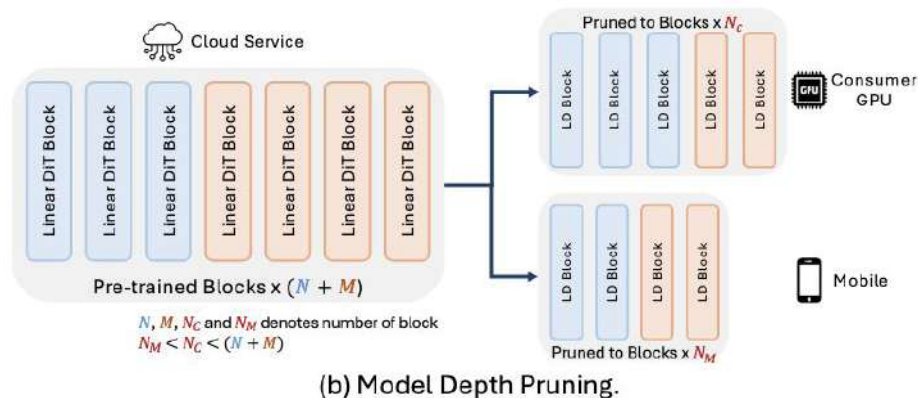


Candles and flowers neatly placed on a table.



Two stuffed bears are dressed in astronaut suits.

Structured Sparsity

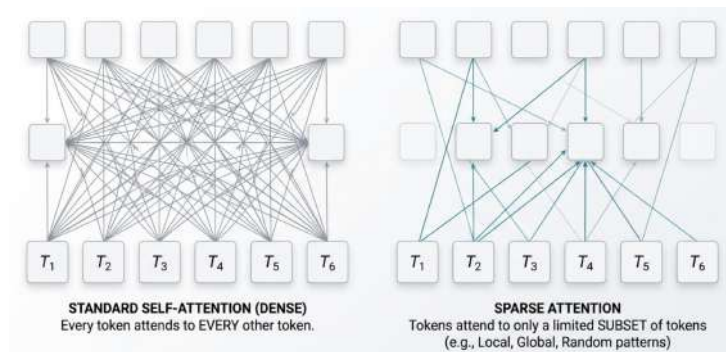
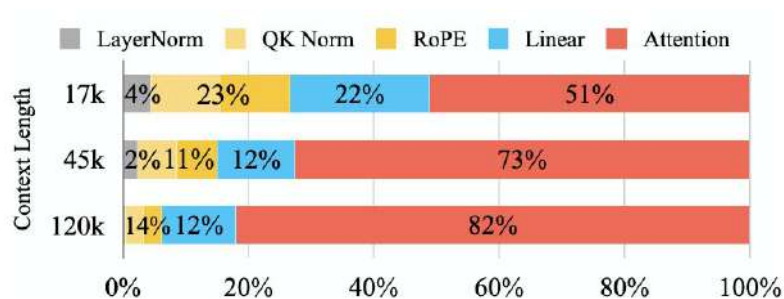


$$BI_i = 1 - \mathbb{E}_{X,t} \frac{\mathbf{X}_{i,t}^T \mathbf{X}_{i+1,t}}{\|\mathbf{X}_{i,t}\|_2 \|\mathbf{X}_{i+1,t}\|_2},$$

Block importance metric

For DiT one can remove the least salient layers and finetune on some dataset to meet memory/compute demands of the given dataset

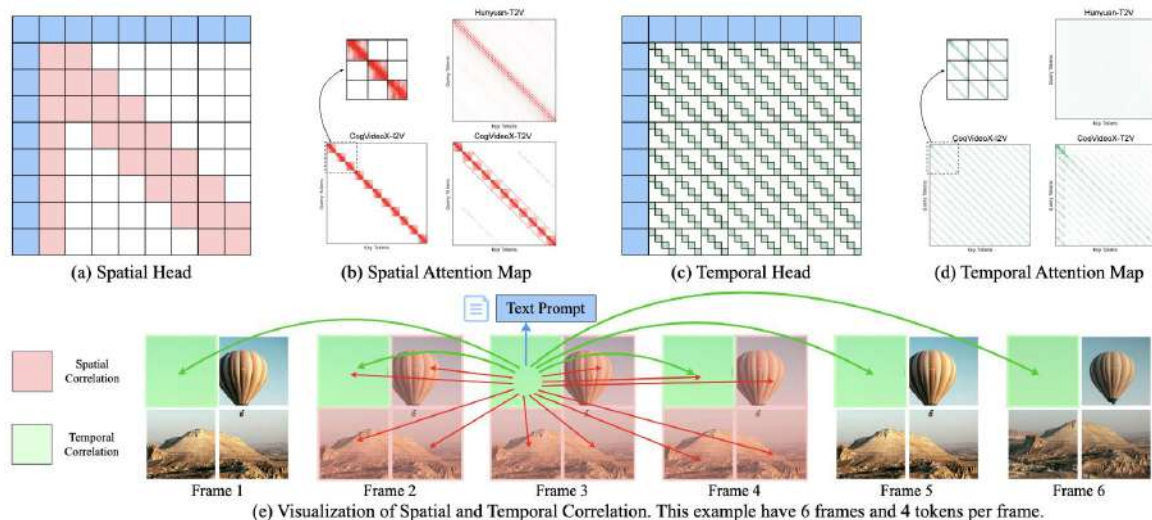
Sparse VideoGen: Accelerating Video Diffusion Transformers with Spatial-Temporal Sparsity



Attention in Video
is a dominant source of latency

At the same time attention is **sparse** and **local**.
Each token strongly attends to a small subset.

Sparse VideoGen: Accelerating Video Diffusion Transformers with Spatial-Temporal Sparsity

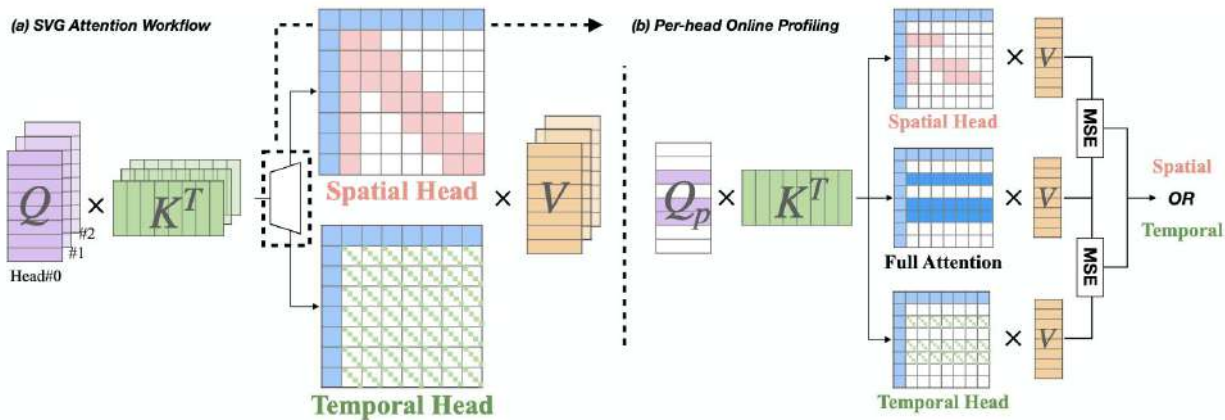


Authors of SVG observed that there are 2 common patterns:

Spatial heads
attending to nearby tokens
with the same frame

Temporal heads
attending to the tokens at
the same location within
different frames

Sparse VideoGen: Accelerating Video Diffusion Transformers with Spatial-Temporal Sparsity



Sparse Video Gen performs online profiling determining whether the given head is spatial or temporal

Sparse VideoGen: Accelerating Video Diffusion Transformers with Spatial-Temporal Sparsity

Table 1. Quality and efficiency benchmarking results of SVG and other baselines.

Type	Method	Quality					Efficiency		
		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	ImageQual $\uparrow$	SubConsist $\uparrow$	FLOPs $\downarrow$	Latency $\downarrow$	Speedup $\uparrow$
I2V	CogVideoX-v1.5 (720p, 10s, 80 frames)	-	-	-	70.09%	95.37%	147.87 PFLOPs	528s	1x
	DiTFastAitA (Spatial-only)	24.591	0.836	0.167	70.44%	95.29%	78.86 PFLOPs	338s	1.56x
	Temporal-only	23.839	0.844	0.157	70.37%	95.13%	70.27 PFLOPs	327s	1.61x
	MInference	22.489	0.743	0.264	58.85%	87.38%	84.89 PFLOPs	357s	1.48x
	PAB	23.234	0.842	0.145	69.18%	95.42%	105.88 PFLOPs	374s	1.41x
	Ours	28.165	0.915	0.104	70.41%	95.29%	74.57 PFLOPs	237s	2.23x
T2V	CogVideoX-v1.5 (720p, 10s, 80 frames)	-	-	-	62.42%	98.66%	147.87 PFLOPs	528s	1x
	DiTFastAitA (Spatial-only)	23.202	0.741	0.256	62.22%	96.95%	78.86 PFLOPs	338s	1.56x
	Temporal-only	23.804	0.811	0.198	62.12%	98.53%	70.27 PFLOPs	327s	1.61x
	MInference	22.451	0.691	0.304	54.87%	91.52%	84.89 PFLOPs	357s	1.48x
	PAB	22.486	0.740	0.234	57.32%	98.76%	105.88 PFLOPs	374s	1.41x
	Ours	29.989	0.910	0.112	63.01%	98.67%	74.57 PFLOPs	232s	2.28x
T2V	HunyuanVideo (720p, 5.33s, 128 frames)	-	-	-	66.11%	93.69%	612.37 PFLOPs	2253s	1x
	DiTFastAitA (Spatial-only)	21.416	0.646	0.331	67.33%	90.10%	260.48 PFLOPs	1238s	1.82x
	Temporal-only	25.851	0.857	0.175	62.12%	98.53%	259.10 PFLOPs	1231s	1.83x
	MInference	23.157	0.823	0.163	63.96%	91.12%	293.87 PFLOPs	1417s	1.59x
	Ours	29.546	0.907	0.127	65.90%	93.51%	259.79 PFLOPs	1171s	1.92x
	Ours + FP8	29.452	0.906	0.128	65.70%	93.51%	259.79 PFLOPs	968s	2.33x

One can achieve ~2x speed-up relative to dense attention without visible quality degradation

CogVideoX-v1.5, 720p, 80 frames, Image-to-Video

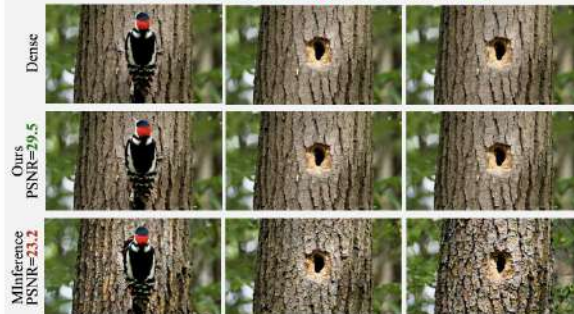
Dense Attention (9 mins $\times$) VS Ours (4 mins $\checkmark$)



Prompt: "A beautiful woman in a blue sari posing in front of a wall"

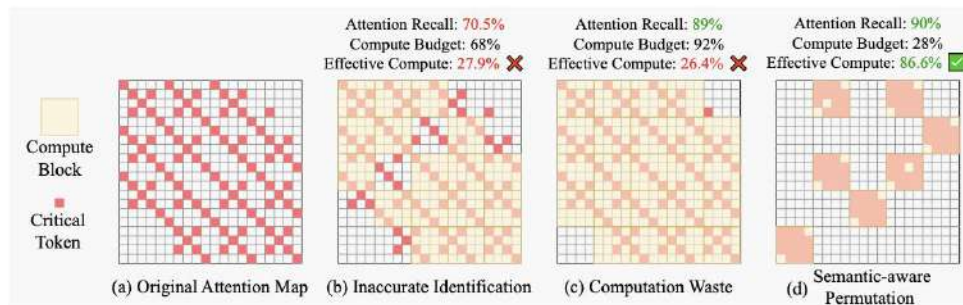
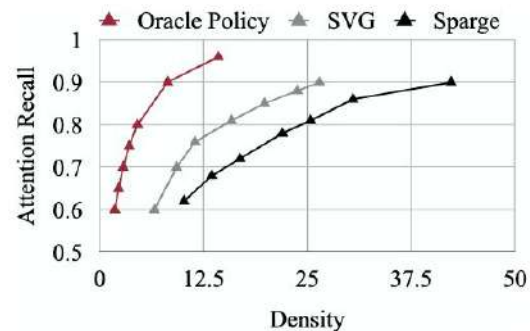
HunyuanVideo, 720p, 128 frames, Text-to-Video

Dense Attention (38 mins $\times$) VS Ours (16 mins $\checkmark$)



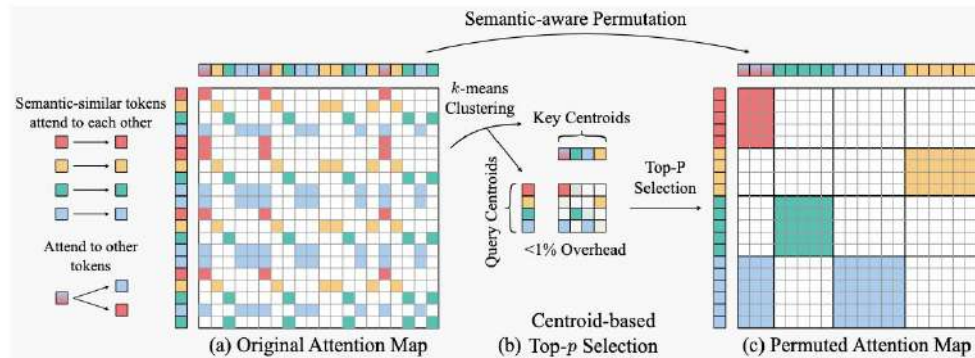
Prompt: "A woodpecker is pecking holes in the tree."

Sparse VideoGen2: Accelerate Video Generation with Sparse Attention via Semantic-Aware Permutation



Spatial & Temporal sparsity does not always accurately represent actual mask sparsity.
More generic sparsity patterns may occur.

Sparse VideoGen2: Accelerate Video Generation with Sparse Attention via Semantic-Aware Permutation



$$S_{ij} = \frac{\text{centroid}(Q_i) \cdot \text{centroid}(K_j)^T}{\sqrt{d_k}}$$

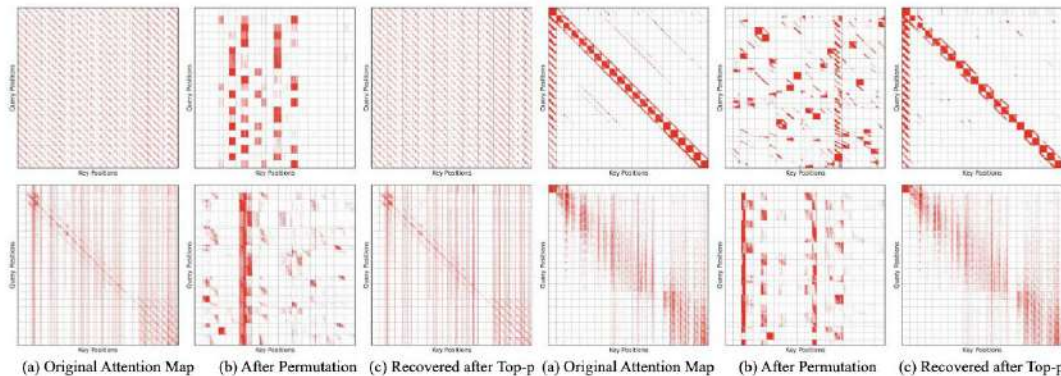
Scores between centroids

$$P'_{ij} = \frac{|K_j| \exp(S_{ij})}{\sum_{k=1}^{C_k} |K_k| \exp(S_{ik})}$$

Taking Top-p scores

Sparse VideoGen2 proposes to cluster tokens into strongly attending groups. For efficient computation these tokens are gathered into blocks.

Sparse VideoGen2: Accelerate Video Generation with Sparse Attention via Semantic-Aware Permutation



One can accurately recover the original attention map via taking the Top-p elements from the permuted attention clusters.

Sparse VideoGen2: Accelerate Video Generation with Sparse Attention via Semantic-Aware Permutation

HunyuanVideo 720P, Text-to-Video

Dense Attention | Latency = 30 min



Sparse VideoGen 2 | PSNR = 30.452 | Latency = 13 min



The shot is vibrant and upbeat, capturing a British white cat with a glossy coat sprinting energetically across a sunlit beach. Its four feet are off the ground, suggesting a sprint caught in suspension. The cat is adorned with stylish, miniature sunglasses that sit snugly on its face, adding a playful and charming touch to its appearance. The sun casts a glistening glow on the gentle waves that lap at the shore, while the sand kicks up slightly under the cat's paws with each bound.

Wan 2.1 720P, Text-to-Video

Dense Attention | Latency = 30 min



Sparse VideoGen 2 | PSNR = 25.808 | Latency = 16 min



Vertical and wide-angle, with ultra-high definition clarity and an artistic, timeless touch, the scene showcases a Japanese princess suspended in the clear blue sky, wearing a beautifully ornate kimono with intricate embroidery and vibrant colors, the embodiment of traditional beauty. Her long, flowing sleeves and elegantly styled hair complete her regal appearance. Behind her, faded but vivid bursts of colorful fireworks sporadically explode, casting a gentle glow. The scene has a dreamy charm.

Thanks for

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}}{\sqrt{d_k}}\right)$$